

Today's Code Changes & .pt File Guide

Quick Answer: What is model.pt?

model.pt does NOT contain a vector. It contains the **trained weights** (parameters) of the CNN — like a brain that has learned. The **64-D z_weather vector** is produced *by running* that model on an input. That's **weather_latent_vector.pt**.

model.pt (the trained brain)

+

ocean_weather_tensor.pt (an ocean snapshot input)

↓ python train_cnn_pipeline.py --mode extract

weather_latent_vector.pt ← this is the actual 64-D z_weather vector

All .pt Files Explained

File	Shape	Size	What it is	Keep?
model.pt	checkpoint dict	129 KB	Trained CNN weights (epoch 49, val_loss=0.0054). This is your deliverable.	✅ YES — give this to teammates
weather_latent_vector.pt	(1, 64)	1.9 KB	The actual 64-D z_weather embedding extracted from one snapshot. This is what the routing pipeline consumes.	✅ YES — example output

File	Shape	Size	What it is	Keep?
<code>training_tensor.pt</code>	<code>(216, 8, 64, 64)</code>	27 MB	All 216 ERA5 time-step snapshots used for training	✓ Keep (needed to retrain)
<code>training_labels.pt</code>	<code>(216, 2)</code>	3.3 KB	Physics labels [R_wave, fuel_rate] for all 216 samples	✓ Keep (needed to retrain)
<code>ocean_weather_tensor.pt</code>	<code>(8, 64, 64)</code>	130 KB	Original single snapshot tensor (pre-existing, used for inference)	✓ Keep (useful for demos)

Code Changes Made Today

1. `train_cnn_pipeline.py` — HEAVILY MODIFIED

Bugs Fixed

- **ReduceLROnPlateau verbose=False** removed (deprecated in PyTorch ≥ 2.2)
- **Silent label fallback** — now raises `FileNotFoundError` if `--labels_path` file doesn't exist
- **std() NaN for N=1** — changed to `correction=0` (population std)
- **80/20 split crash with tiny datasets** — added minimum-sample guard + graceful no-validation path

Features Added

- **NetCDF ingestion engine** (Section 2.5, ~300 lines):
 - Auto-discovers ERA5/CMEMS variable name aliases (handles `VHM0`, `swh`, `significant_wave_height`, etc.)
 - Detects `lat/latitude/nav_lat`, `lon/longitude/nav_lon` dimension names automatically
 - Tiles any grid into 64×64 patches with configurable `--nc_stride`
 - Uses `ds.sizes` instead of `ds.dims` (avoids xarray FutureWarning)
 - Zero-pads missing channels with a warning

- `--nc_path` flag — point at a `.nc` file or folder
- `--nc_stride` flag — patch overlap control (default 32 = 50% overlap)
- `--nc_time_step` flag — use every Nth time-step

Priority chain in `train()`

1. `--nc_path` → real NetCDF data (preferred)
 2. `--data_path` → pre-built `.pt` tensor
 3. `synthetic` → auto-generated fallback
-

2. `ingestion_pipeline.py` — MODIFIED

Features Added

- `run_all()` function — loops over all N time-steps, processes each through the full pipeline
- `_physics_labels()` function — computes Bretschneider `R_wave` / `fuel_rate` from real field values
- `--all_timesteps` CLI flag — triggers `run_all()` instead of single-step `run()`
- `--labels_output` CLI flag — where to save the physics labels tensor
- `run()` updated — accepts `time_index` parameter (not just always index 0)

What it now produces with `--all_timesteps`

`training_tensor.pt` (216, 8, 64, 64) ← all ERA5 time-steps normalised

`training_labels.pt` (216, 2) ← `R_wave` + `fuel_rate` per step

3. `cnn_diagnostics.py` — NEW FILE

Standalone visual diagnostic dashboard. Run with:

```
python cnn_diagnostics.py
```

Produces `cnn_diagnostics.png` with 3 panels:

Panel	Contents
A — Input Fields	8-channel heatmap of a real ERA5 snapshot (what the CNN sees)
B — Embedding Space	PCA 2-D projection of all 216 z_weather vectors, coloured by Hs, wind speed, R_wave
C — Accuracy	Predicted vs actual scatter ($R^2=0.995$) + intrinsic dimensionality plot

Your Full Workflow (reproducible end-to-end)

Step 1: Convert all ERA5 time-steps to training tensors

```
python ingestion_pipeline.py --all_timesteps
```

Step 2: Train the CNN on real data

```
python train_cnn_pipeline.py --mode train \
```

```
--data_path training_tensor.pt \
```

```
--labels_path training_labels.pt \
```

```
--epochs 50
```

Step 3: Extract z_weather vector from any snapshot

```
python train_cnn_pipeline.py --mode extract \
```

```
--data_path ocean_weather_tensor.pt \
```

```
--output_path weather_latent_vector.pt
```

Step 4: Generate diagnostic dashboard

```
python cnn_diagnostics.py
```